

基于 Qwen3-VL 的 TextVQA 单卡基线复现与改进探索

姓名

May 17, 2026

1 目标

本实验围绕 TextVQA，使用 `Qwen/Qwen3-VL-2B-Instruct` 作为基础模型，完成在满足训练预算与推理约束的前提下，对 baseline 进行改进。

2 环境

2.1 硬件与系统

- 操作系统: Windows
- GPU: 4060
- GPU 数量: 1

2.2 软件环境

实验使用项目本地 Conda 环境，路径为：

```
.\.conda-env
```

3 模型与方法

3.1 基础模型

本实验采用的基础模型为：

`Qwen/Qwen3-VL-2B-Instruct`

该模型属于视觉语言生成模型，整体由视觉编码器、语言模型主干以及多模态融合机制组成。输入为图像与文本提示，输出为生成式文本答案。

3.2 Baseline 微调方式

baseline 采用 LoRA 进行轻量微调，具体方式为：

- 冻结视觉塔参数；
- 在语言模型中的若干线性投影层上插入 LoRA；
- 使用生成式训练目标进行 TextVQA 微调。

3.3 baseline 配置

配置项	取值
基础模型	Qwen/Qwen3-VL-2B-Instruct
数据集	lmms-lab/textvqa
最大训练样本数	40960
最大训练步数	1024
最大训练时长	3600 秒
batch size	1
梯度累积步数	8
学习率	1×10^{-5}
LoRA rank	16
LoRA alpha	32
LoRA dropout	0.05
LoRA 目标模块	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj

表 1: 当前单卡 baseline 配置

4 数据处理与训练流程

实验使用的数据源为：

lmms-lab/textvqa

训练流程包括以下几个阶段：

1. 使用 `prepare` 脚本对原始数据进行处理；
2. 基于指定 seed 构造 prepared 数据集；
3. 使用 LoRA 对基础模型进行训练；
4. 将 LoRA adapter 与基础模型合并；
5. 使用评测脚本在 TextVQA 验证集上进行评估。

prepared 数据按种子分别保存为:

```
data/prepare_textvqa_qwen3vl_seed1
data/prepare_textvqa_qwen3vl_seed2
data/prepare_textvqa_qwen3vl_seed3
```

5 适配与修复

为了使仓库能够在当前单卡 Windows 环境下稳定运行，完成了以下适配工作：

- 搭建项目本地运行环境并修复依赖问题；
- 为 Windows 新增 `run_prepare.ps1`、`run_train.ps1`、`run_merge_lora.ps1`、`eval_qwen.ps1`；
- 修复无效代理变量导致的 Hugging Face 下载失败问题；
- 修改 `prepare_textvqa.py`，使其支持直接从 Hugging Face 数据集读取；
- 修复 `lmms-eval` 中 TextVQA 任务定义残留的实验室路径；
- 将 Windows 下 `dataloader_num_workers` 自动置为 0，以避免多进程权限错误；
- 修复 `merge` 脚本在不同 seed 下路径复用不正确的问题；
- 改进评测脚本输出，使当前评测的模型路径更加清晰可见。

6 复现结果

6.1 基座模型结果

未微调的基座模型在 TextVQA 验证集上的结果为：

$$\text{exact_match} = 69.89\%$$

6.2 单卡 LoRA baseline 结果

seed1 结果为：

$$\text{exact_match} = 71.012\%$$

seed2 结果为：

$$\text{exact_match} = 70.906\%$$

seed3 结果为：

$$\text{exact_match} = 70.988\%$$

6.3 结果汇总表

模型 / 设置	Seed 1	Seed 2	Seed 3
基座模型（无微调）	69.89	—	—
LoRA baseline（单卡）	71.012	70.906	70.988

表 2: 当前主要结果汇总（exact match, 单位：%）

单卡 LoRA baseline 平均值：

70.969%

7 后续改进

7.1 方案一：调整 LoRA 超参

LoRA 的 rank、alpha 和 dropout 会直接影响可训练容量、训练稳定性以及泛化效果。

计划设置：

- lora_r: 32
- lora_alpha: 64
- lora_dropout: 0.03

结果：

exact_match = 71.076%

没有明显提升，当前 LoRA 配置可能已经较为接近局部最优。

7.2 方案二：引入 OCR token

当前 baseline 中：

`use_ocr_tokens: false`

这意味着模型没有显式使用数据集中已有的 OCR token 信息。启用 OCR token 有望增强模型对图中文字的感知能力。但是可能导致 1 小时内无法走完 1024 步导致不满足要求，因此先尝试引入 8 个 OCR token 并延长时间限制完整训练，观察结果，再决定后续是否调整 token 数量、步数、学习率等参数。

计划设置：

- `use_ocr_tokens: true`
- `max_ocr_tokens: 8`

结果:

`exact_match = 72.754%`

结果有提升, 但训练时发现单张 4060 一小时内只能完成约 800 步迭代。接下来使用更少的步数和更大的学习率调整训练配置, 观察是否能在 1 小时内完成训练并获得更好的结果。

计划设置:

- `use_ocr_tokens: true`
- `max_ocr_tokens: 8`
- `max_steps: 768`
- `learning_rate:  $1.5 \times 10^{-5}$`

在该配置下本地训练时长为 52 分 54 秒。

结果:

`exact_match = 72.5%`

7.3 方案三: 提示词微调 (Prompt Tuning)

改动动机: 该方法参数量较小, 推理时只额外引入少量前缀 token, 有望在满足测试时延迟和 FLOPs 约束的前提下提升模型表现。

计划设置:

- 虚拟 token 数量: 16
- 初始化方式: TEXT 初始化, 初始化文本为 `Answer TextVQA questions using a single word or short phrase.`

结果:

`exact_match = 68.502%`

该配置下 prompt tuning 在 40 分钟左右即可训完, 预算时间还有盈余, 可以继续优化。但是考虑到 VLM 输入的复杂性, prompt tuning 的小容量和偏向文本侧的性质可能使其表现不佳, 因此不做后续改动。

7.4 方案四: 轻量 Adapter

计划设置:

- 方法类型: Bottleneck Adapter
- 冻结策略: 冻结视觉塔与基础模型主体参数, 仅训练 adapter 参数
- `adapter hidden dimension: 32`
- `adapter dropout: 0.05`

- adapter activation: `gelu`
- 插入层范围: 语言模型最后 8 层

结果:

`exact_match = 69.472%`

该配置下训练时长约 35 分钟, 但结果不如 LoRA baseline。由于仍有富余训练时间, 因此考虑提高 adapter 维度和插入层数, 并减小随机丢弃概率, 观察是否能提升结果。

计划设置:

- adapter hidden dimension: 64
- adapter dropout: 0.03
- adapter activation: `gelu`
- 插入层范围: 语言模型最后 12 层

结果:

`exact_match = 71.258%`

8 最终方案

经过前述实验比较, 方案二中“LoRA + OCR token”的单种子结果最好。其中, 完整 1024 步 OCR 训练取得了 72.754% 的 exact match, 但单张 4060 难以在 1 小时内完成; 将训练步数调整为 768、学习率调整为 1.5×10^{-5} 后, 本地训练时长约为 52 分 54 秒, 仍取得 72.5% 的 exact match。因此, 最终选择该配置作为三种子实验方案。

最终采用的配置:

- 基础方法: LoRA 微调
- OCR 设置: 使用 TextVQA 数据集自带 OCR token
- `use_ocr_tokens: true`
- `max_ocr_tokens: 8`
- `max_steps: 768`
- `max_train_seconds: 3600`
- 学习率: 1.5×10^{-5}
- 训练 seed: 1、2、3

后续执行流程:

1. 分别对 seed 1、2、3 使用 OCR 配置重新准备数据;

2. 使用相同的 LoRA + OCR 配置完成三次训练；
3. 分别将三个 seed 的 LoRA adapter 与基座模型合并；
4. 使用对应的 OCR 评测任务进行验证集评估；
5. 记录三个 seed 的 exact match，并计算平均值作为最终结果。

结果记录：

方法	Seed 1	Seed 2	Seed 3	Mean
LoRA baseline	71.012	70.906	70.988	70.969
LoRA + OCR token	72.500	72.634	72.808	72.647

表 3: 最终 LoRA + OCR 三种子实验结果记录 (exact match, 单位: %)

9 总结

本文围绕 TextVQA 任务,在单卡 Windows 环境下完成了基于 Qwen/Qwen3-VL-2B-Instruct 的完整实验流程搭建,包括数据准备、训练、模型合并和评测。实验过程中对原始仓库进行了 Windows 适配,补充了 PowerShell 脚本,并修复了 Hugging Face 数据读取、模型评测路径、不同 seed 输出目录管理等问题,使得实验流程能够在本地稳定复现。

在 baseline 复现阶段,未微调的基座模型在 TextVQA 验证集上的 exact match 为 69.886%。单卡 LoRA baseline 在三个 seed 上分别取得 71.012%、70.906% 和 70.988%,平均值为 70.969%。在此基础上,本文进一步尝试了多种轻量化改进方案,包括 LoRA 超参数调整、引入 OCR token、Prompt Tuning 和轻量 Adapter。

实验结果表明,单独调整 LoRA 超参数提升有限;Prompt Tuning 虽然参数量极小、训练速度较快,但在当前 VLM 场景下表达能力不足,结果低于 LoRA baseline;轻量 Adapter 在扩大 adapter 维度和插入层数后有所提升,但整体仍弱于 OCR 方案。相比之下,引入 TextVQA 数据集自带 OCR token 的 LoRA 方法提升最明显。

最终采用的方案为 **LoRA + OCR token**。该方案在输入 prompt 中显式加入最多 8 个 OCR token,并保持 LoRA 微调结构不变。在前期探索中,完整 1024 步 OCR 训练的 seed 1 结果达到 72.754%,但训练时间超过 1 小时预算。因此,最终方案将训练步数调整为 768,并将学习率设为 1.5×10^{-5} ,以满足训练时间约束。

在最终预算内配置下,LoRA + OCR token 三个 seed 的 exact match 分别为 72.500%、72.634% 和 72.808%,平均值为 72.647%。相比单卡 LoRA baseline 平均值 70.969%,提升约 1.68 个百分点;相比未微调基座模型 69.886%,提升约 2.76 个百分点。

总体来看,OCR token 能够有效补充 TextVQA 任务中图中文字信息,是本实验中收益最稳定、效果最明显的改进方向。最终提交将采用 LoRA + OCR token 的三种子平均结果作为主要结果,并与原始 LoRA baseline、Prompt Tuning 和 Adapter 实验结果进行对比分析。